НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-242
Балик А.В.

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Kyiv	Київ
eval_count	274	5
Час, с	17,11	7,5
Токенів/с	23,19	0.66

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1306.4	133.8
Час, с	63.68	3.89

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками. Та відповідь «55 грн», бо модель інколи вважає ручку та зошит однаковими зав ціною.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: міркувальна модель може витрачати додаткові токени на внутрішні міркування: аналіз запитання, перевірку можливих відповідей і формування остаточного висновку. У відповіді qwen3:4b це може відображатися у полі thinking. Тому не всі згенеровані токени обов’язково потрапляють до кінцевої відповіді

користувачу. Для простого фактичного питання такі додаткові обчислення не дали помітної переваги в точності.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі особливо корисні для складних математичних задач, логічних головоломок, програмування, аналізу даних і задач, де потрібно послідовно враховувати кілька умов. Додаткове опрацювання може допомогти їм перевіряти проміжні висновки та знаходити помилки.

Для простих фактичних запитань, перекладу, коротких пояснень і нескладного форматування використання міркувальної моделі часто невиправдане. Інструктивна модель може надати правильну відповідь швидше, витративши менше токенів. Проте результат залежить від конкретного запиту та моделей.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі зазвичай додатково навчають виконувати команди користувача, дотримуватися формату відповіді та бути корисними в діалозі. Для цього можуть застосовувати навчання на прикладах запитів і відповідей та методи оптимізації поведінки моделі.

Для міркувальних моделей додатково приділяють увагу розв’язанню складних задач і побудові послідовних висновків. Залежно від підходу, це може включати спеціально підібрані задачі, приклади міркувань, навчання з підкріпленням та оцінювання правильності результатів. Точні методи навчання конкретних qwen3:4b і llama3.2:3b не можна встановити лише за результатами цього експерименту.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні. Більша кількість токенів сама по собі не свідчить про вищий інтелект або кращу якість моделі. Додаткові токени можуть використовуватися для корисних міркувань, але можуть також містити зайві пояснення, повторення або нерелевантний текст.

У моєму експерименті обидві моделі правильно відповіли на просте фактичне питання, хоча qwen3:4b використала значно більше токенів. На задачі про ручку міркувальна модель дала 5 правильних відповідей із 5, а інструктивна — 1 із 5. Це свідчить про перевагу qwen3:4b саме на цій задачі, але для загального висновку про якість моделей потрібно перевірити більше різноманітних завдань.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: За результатами вимірювань на простому питанні швидкість qwen3:4b становила 23,19 токена/с, а llama3.2:3b — 0,66 токена/с. Отже, у цьому запуску qwen3:4b генерувала токени приблизно у 35 разів швидше.

Такий результат не обов’язково означає, що міркувальні моделі загалом швидші. На швидкість впливають розмір моделі, її архітектура, квантизація, апаратне забезпечення, завантаження моделі в пам’ять, довжина контексту та характер генерації. Крім того, вимірювання одного запуску можуть бути нестабільними.

Висновок за завданням 1

Експеримент показав, що швидкість генерації токенів і загальний час відповіді — це різні показники. На простому фактичному питанні обидві моделі відповіли правильно: qwen3:4b — Kyiv, а llama3.2:3b — Київ. Проте міркувальна модель згенерувала 274 токени, тоді як інструктивна — лише 5, тобто у 54,8 раза більше. Загальний час становив 17,11 с проти 7,5 с. Це свідчить про надмірні витрати міркувальної моделі на простому завданні.

На задачі про ручку й зошит ситуація була іншою: qwen3:4b дала правильну відповідь у 5 випадках із 5, тоді як llama3.2:3b — лише в 1 випадку з 5. Середній час виконання становив відповідно 63,68 с і 3,89 с. Отже, міркувальна модель показала кращий результат на логічній задачі, але витратила значно більше часу. Практичний висновок: модель потрібно обирати залежно від типу завдання. Для агента з 8 послідовними викликами за наведеними середніми показниками загальний час становив би приблизно 509,44 с проти 31,12 с. Тому міркувальна модель може бути надто повільною для налагодження агента, якщо кожен виклик потребує тривалого міркування.

Під час одного із запусків також виникла помилка prediction aborted, token repeat limit reached. Вона означає, що генерацію було перервано через надмірне повторення токенів. Такий запуск не слід вважати успішною відповіддю моделі, але й даних часу й токенів я не отримав, тому не враховував її в 5 запусків.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Завдання 2 варіанту. Питання користувача (user): Поясни різницю між стеком і чергою. Запуск python hello.py . Запуск програми з різними system-повідомленнями для обох моделей.

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	~ 1900	Українська з домішками англійської	16	Ні, обидва терміни взагалі неправильно описала
llama3.2:3b	українською, 2 речення	221	Українська	2	Не зовсім. Описала як не технічні, а загальні поняття.
qwen3:4b	лаконічний асистент	91	Українська	2	Так, дуже лаконічно
qwen3:4b	українською, 2 речення	159	Українська	2	Так, ідеально правильно

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	423	473
llama3.2:3b	українською, 2 речення	54	93	147
qwen3:4b	лаконічний асистент	52	2039	2091
qwen3:4b	українською, 2 речення	61	631	692

Відповіді на питання (крок 6):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі в наведених коротких відповідях надали по два речення. Проте qwen3:4b із system-повідомленням «українською, 2 речення» сформулювала правильну відповідь лаконічніше.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Так, у відповіді llama3.2:3b з'явилися англійські слова та некоректні змішані формулювання.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: У llama3.2:3b кількість вхідних токенів зросла з 50 до 54, а в qwen3:4b — з 52 до 61. Це пов'язано зі зміною тексту system-повідомлення, оскільки модель враховує його разом із запитом користувача; кількість токенів залежить від токенізації тексту.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Модель qwen3:4b може генерувати додаткові токени для внутрішнього міркування, які не відображаються у фінальній відповіді. Саме тому набагато більші показники токенів.

Відповіді чат-боту (крок 7):

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Моделі відрізняються архітектурою, навчальними даними, налаштуванням для виконання інструкцій і здатністю враховувати контекст. Тому одна модель може точніше виконувати вимоги до мови та довжини, а інша — відхилятися від них.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає загальні правила поведінки моделі, тоді як user-повідомлення містить конкретний запит користувача. Зазвичай system-повідомлення має вищий пріоритет, тому за конфлікту інструкцій модель повинна насамперед дотримуватися системних правил.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Міркувальна модель може витрачати додаткові токени на аналіз завдання перед формуванням фінальної відповіді. Проте велика різниця в лічильниках не доводить сама по собі наявність внутрішнього міркування: необхідно перевірити формат відповіді API та спосіб підрахунку токенів.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Зазвичай модель надасть перевагу system-інструкції, оскільки вона має вищий пріоритет. Наприклад, якщо system вимагає відповідати українською, а user просить англійською, модель повинна відповісти українською, якщо немає інших інструкцій вищого пріоритету.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі зазвичай мають менше параметрів і можуть гірше засвоювати складні закономірності мови та пріоритети інструкцій. Водночас результат залежить не лише від розміру, а й від якості навчання, мовного складу даних і налаштування моделі.

Висновок за завданням 2

System-повідомлення є корисним засобом керування поведінкою мовної моделі: воно допомагає визначити мову, стиль, довжину та формат відповіді. Проте воно не гарантує стовідсоткового виконання інструкцій, особливо для невеликих моделей або складних завдань. Якщо модель ігнорує вимоги, варто сформулювати їх чіткіше, використовувати конкретні обмеження та перевіряти результат

програмно. Для підвищення надійності можна також обмежувати максимальну кількість вихідних токенів і перевіряти структуру відповіді після генерації.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Питання (QUESTION) для 2 варіанту: Поясни різницю між стеком і чергою. Коротко. З використанням ключу для роботи з хмарною моделлю, було порівняно з локальною.

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	3.05	1.15
completion_tokens	43	161
Довжина відповіді, символів	111	390
Мова відповіді	Українська, але текст не зрозумілий	Українська
Фактичних помилок у відповіді	1, фактично уся відповідь це помилка	0

Відповіді на питання (крок 5):

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У моєму випадку потрібно було змінити приблизно 2–3 рядки конфігурації: адресу base_url, і назву моделі, а також API-ключ в .env .

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель gemini-3.5-flash-lite надала повнішу й точнішу відповідь. Вона правильно пояснила принципи LIFO для стеку та FIFO для черги й навела аналогії зі стопкою тарілок і чергою в магазині, тоді як локальна модель згенерувала незрозумілий текст про поляризоване світло:

«Різниця між стеком та чергою полярної світилки поляризуючого світла, поляризованного світла, а потім, знову ж так, світла.»

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, локальна модель використала незрозуміле формулювання та не пояснила різницю між стеком і чергою. Її відповідь була фактично українською за окремими словами, але не мала ніякого змісту.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарний провайдер використовує потужні сервери, спеціалізовані прискорювачі та оптимізовану інфраструктуру для генерації тексту, тоді як у мене просто мій ноутбук. У цьому експерименті ці переваги компенсували мережеву затримку: хмарна модель відповіла за 1,15 секунди, а локальна — за 3,05 секунди.

Питання до чат-бота (крок 6):

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п’ять питань:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI API набув широкого поширення завдяки популярності своїх моделей і зрозумілому інтерфейсу для розробників. Сумісність із цим форматом дозволяє провайдерам повторно використовувати наявний код і бібліотеки, спрощуючи перехід між різними моделями.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Більша кількість параметрів може покращувати здатність моделі розуміти складні запити, міркувати й дотримуватися інструкцій, але не гарантує кращого результату. На якість також впливають навчальні дані, архітектура, налаштування моделі, системні інструкції та спосіб формулювання запиту.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарні сервіси можуть використовувати потужні GPU або спеціалізовані прискорювачі й оптимізовані системи обробки запитів. Локальна модель залежить від продуктивності комп'ютера, доступної пам'яті та швидкості обчислень, тому навіть без мережевої затримки може працювати повільніше.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Надсилаючи запит до хмарного API, користувач передає його вміст зовнішньому провайдеру, де він може оброблятися та зберігатися відповідно до умов сервісу. У комерційних проєктах застосовують видалення персональних даних, шифрування, контроль доступу, політики зберігання та перевірку умов використання API.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це обмеження на кількість запитів або токенів за певний проміжок часу. Розробники керують навантаженням за допомогою черг, кешування, повторних спроб із затримкою та переходу на інший доступний тариф або провайдера; ці методи не скасовують встановлених обмежень.

Висновок за завданням 3

Я обрав би локальну модель, коли важливі приватність даних, автономна робота без інтернету та відсутність оплати за кожен запит. Хмарну модель доцільніше використовувати, коли потрібні вища якість відповідей, складніший аналіз і швидка обробка завдань, а передавання даних провайдеру є прийнятним. Отже, вибір залежить від балансу між якістю, швидкістю, вартістю та конфіденційністю.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Для 2 варіанту:

Короткий промпт: Столиця Японії?

Довгий промпт: Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.

Були виконані запити для локальної і хмарної моделей і порівняні показники. Перед заміром був проведений прогрів, потім взято середню з 3 запусків.

Таблиця 6: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.06	2.40	13.3
локальна	llama3.2:3b	середній	2.07	4.30	85.4
локальна	llama3.2:3b	довгий	2.09	13.69	124.7
хмарна	gemini-3.5-flash-lite	короткий	0.64	0.70	38.6
хмарна	gemini-3.5-flash-lite	середній	0.60	1.06	383.9
хмарна	gemini-3.5-flash-lite	довгий	0.69	4.48	861.8

Таблиця 7: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.06	2.05
Всього, с	2.50	2.34

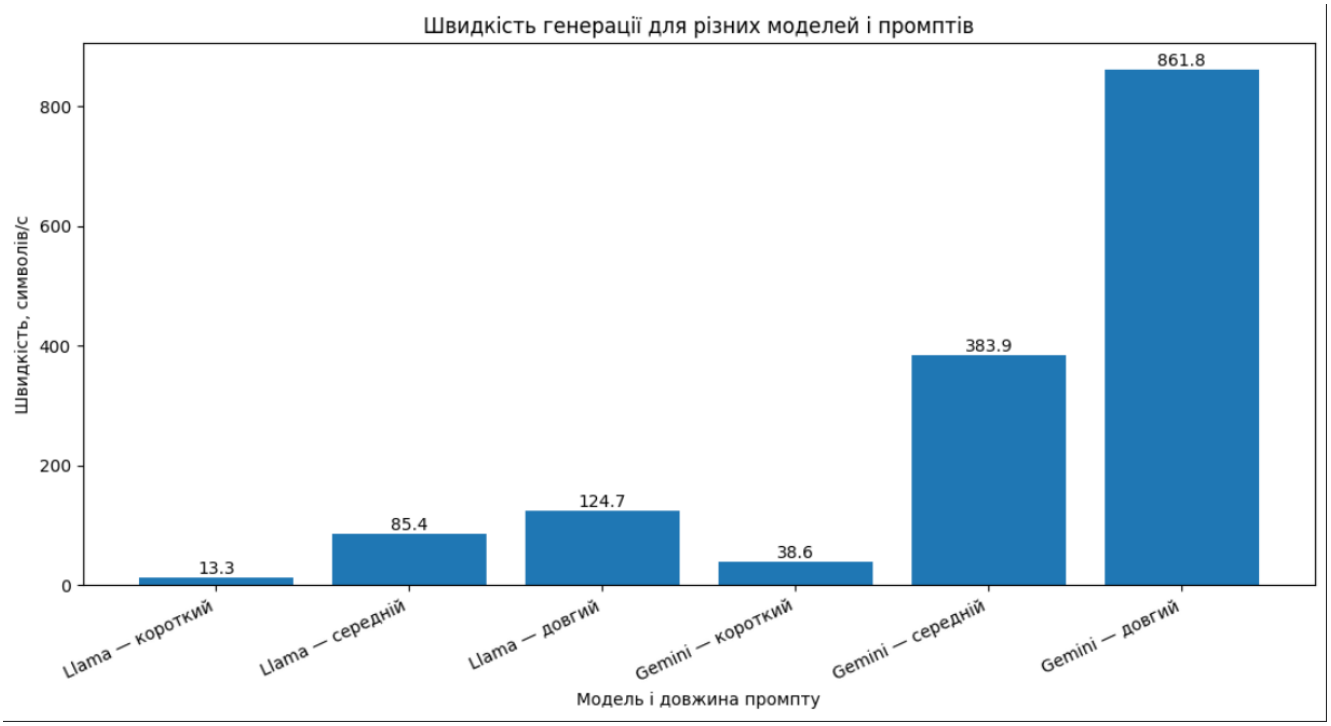


Рисунок 1. Швидкість генерації для різних моделей

Відповіді на запитання (крок 6):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT має хмарна модель gemini-3.5-flash-lite: її показники становлять 0,60–0,69 с, тоді як локальна llama3.2:3b показує 2,06–2,09 с. Це може пояснюватися оптимізованою серверною інфраструктурою та потужнішим обладнанням хмарного провайдера.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Так, збігається. Хмарна модель має кращий TTFT і водночас вищу швидкість генерації для всіх трьох промптів: наприклад, для довгого промпту — 861,8 символа/с проти 124,7 у локальної моделі і 0.69 с у хмарної проти 2.09 с у локальної. TTFT вимірює час до першого токена, а символи/с — швидкість подальшої генерації тексту.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: Для короткого промпту хмарна модель генерувала приблизно у 2,9 раза швидше, а для довгого — приблизно у 6,9 раза швидше. Отже, у цих вимірюваннях перевага хмарної моделі за швидкістю генерації збільшилася зі зростанням обсягу відповіді.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Одного запуску недостатньо для надійного висновку: час залежить від завантаження комп'ютера, мережі та стану сервера, тому варто виконати кілька повторів і порівняти їхню медіану. Для великого промту була особливо величезна різниця: від 4 до 20+ секунд.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту модель може завантажувати ваги з диска в оперативну пам'ять або відеопам'ять і ініціалізувати потрібні компоненти. Під час теплого старту модель уже завантажена, тому повторний запит може початися швидше.

Відповіді чат-боту (крок 7):

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: TTFT (Time To First Token) — це час від надсилання запиту до отримання першого токена відповіді. За цей час система обробляє запит, виконує обчислення моделлю та, для хмарного API, враховує мережеву затримку; цей показник важливий для оцінки того, наскільки швидко користувач побачить першу відповідь.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Мережевий обмін і очікування на сервер можуть збільшити TTFT хмарної моделі. Водночас потужні прискорювачі та оптимізована серверна інфраструктура дають їй змогу генерувати наступні токени швидше, ніж комп'ютер користувача.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це запуск, коли модель ще не завантажена в пам’ять і системі потрібно підготувати її до роботи. Під час теплого старту модель уже може бути в пам’яті, тому час початку обробки запиту скорочується.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана кількох запусків зменшує вплив випадкових затримок і нетипово повільних або швидких результатів. Вона дає надійнішу оцінку типового часу роботи, ніж один-єдиний запуск.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Під час обробки вхідного тексту модель може опрацьовувати багато токенів паралельно. Генерація відповіді зазвичай відбувається послідовно: кожен наступний токен залежить від попередніх, тому створення довгого тексту займає більше часу.

Висновок за завданням 4

Я переконався як значення TTFT і швидкість генерування впливають на відповідь. Кращий TTFT не означає що модель буде швидшою. Потужні хмарні моделі мають набагато швидшу швидкість генерації і загальний час, хоча пакети летять за пів світу. Для інтерактивного чат-бота важливими є TTFT і швидкість генерації, оскільки користувач очікує швидко побачити першу відповідь і комфортно читати її продовження. Тоді як для фонової обробки великих обсягів тексту важливішими можуть бути загальна пропускна здатність і час завершення завдання, адже невелика затримка перед першою відповіддю не настільки критична.

Завдання 5. Оцінювання якості моделей за рубрикою

Було оцінено 3 моделі за результатами відповідей на 6 завдань і підраховані бали, які приблизно показують відносне поняття «якості» моделі. Також переконалися у недетермінованості моделей.

Для 2 варіанту:

Тема	Транспорт
Завдання 1 (факт)	Рік відкриття Київського метрополітену
Завдання 2 (формат)	JSON: city, year, lines_count для Київського метро
Завдання 3 (робота з текстом) взятий з вікіпедії	За 2015 рік загальний пасажирообіг на внутрішніх і міжнародних рейсах становив 4,6 млн осіб[1]. За 2015 рік повітряним транспортом було перевезено 37,7 млн тонно-кілометрів вантажів (без врахування багажу пасажирів)[1]. Літаками в Україні перевозять переважно пасажирів. За середньою відстанню перевезення одного пасажира повітряний транспорт в Україні посідає перше місце і перевищує залізничний транспорт більш, як у 10 разів. Пасажирським перевезенням в Україні характерна сезонність. Основний пасажиропотік припадає на літо. Діють повітряні лінії до Франкфурта, Парижа, Торонто, Нью-Йорка, Лондона, Тель-Авіва. Розширюються зв'язки з країнами Близького Сходу, Китаєм.
Завдання 4 (код)	total_weight(items) — сума ваг зі списку словників
Завдання 5 (логіка)	120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів?
Завдання 6 (українська)	Чим контейнерне перевезення відрізняється від навалочного

Таблиця 8: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	2	2
2. Формат	1	2	2

3. Стиснення	0	2	2
4. Код	1	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	2	11	12

Таблиця 9: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Так	Ні	Ні
3. Стиснення	Ні	Ні	Ні
4. Код	Так	Ні	Ні
5. Логіка	Ні	Ні	Ні
6. Українська	Ні	Так	Ні

Авторитетне джерело для 1 завдання:

https://www.bbc.com/ukrainian/blogs/2015/11/151103_kyiv_metro_blog_she

Коментарі до таблиць: в 1 завданні правильна відповідь 1960, тоді як llama3.2:3b надала дві різні неправильні. В 2 завданні в усіх коректний JSON, але один зайвий аргумент в одному прогоні llama. В 3 llama втратила числа. В 4 llama помилилася з кодом в одному з прогонів. В 5 llama почала галюцинувати і неправильно порахувала або взагалі не туди пішла. Тут правильну відповідь 2 інші моделі правильно сказали. В 6 llama знову вигадала багато некоректних термінів, а в 2 прогоні взагалі речення не мають змісту. Тоді як qwen загалом правильно, але присутні русизми і кальки. Gemini все ідеально виконала.

Відповідь на запитання (крок 6):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала хмарна модель Gemini 3.5 Flash Lite — 12 із 12, тоді як Qwen 3 4B отримала 11, а Llama 3.2 3B — 2 бали. Основна різниця виникла в завданнях на стислий переказ, логічні обчислення та якість української мови, де Llama припускалася суттєвих помилок в кожному завданні, а інші майже ідеально.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Розійшлися в завданнях 2 і 4 для llama, і 6 для qwen, тоді як для Gemini розбіжностей немає. Це свідчить про стабільнішу якість Gemini на цих прикладах, хоча двох прогонів недостатньо, щоб зробити остаточний висновок про надійність моделей загалом, але можемо переконатися що відповіді не детерміновані.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Усі виконали, неочікувано.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, llama дуже багато помилялася і переходила то на англійську, то на російську, то вигадувала і комбінувала мови.

Приклад провалу (крок 7).

Запит:

"Чим контейнерне перевезення відрізняється від навалочного>. Поясни українською, два абзаци."

Відповідь llama для 6 завдання:

"Відрізь від навігаційного перевезення лежить в контейнерному перевезенні в тому, що таке перевезення не носить хронічного характеру. \n\nКонтейнерне перевезення відбувається з допомогою контейнера для зберігання та перевезення вантажі. Втім контейнерне перевезення не відносить до навалочного."

Правильна (на прикладі gemini):

"**Контейнерне перевезення** — це спосіб транспортування вантажів у стандартних металевих ємностях (контейнерах), які захищають вміст від погодних умов, крадіжок та пошкоджень. Головна перевага цього методу полягає в універсальності: один і той самий контейнер можна легко перевантажувати з корабля на потяг чи вантажівку без перепакування самого вантажу. Це значно прискорює логістику та знижує витрати на обробку найрізноманітніших товарів — від одягу до електроніки.\n\nНатомість **навалочне (або насипне) перевезення** застосовується для сипучих вантажів безтарного типу, таких як зерно, вугілля, руда, щебінь чи цемент. Вони транспортуються безпосередньо у трюмах суден, у вагонах-хоперах або самоскидах і завантажуються за допомогою спеціальних конвеєрів чи насосів. Такий спосіб є найефективнішим для масових партій сировини, але він вимагає спеціалізованої інфраструктури для перевалки й робить вантаж більш вразливим до вологи та втрат під час транспортування."

Пояснення чому: llama модель працює без розмірковування тому відповіді часто є не зовсім або взагалі не правильними. Тут модель нібито розділила на два абзаци і дала якусь відповідь, але вона зовсім не зрозуміла. Помилки з українською мовою у цієї моделі дуже часті, мабуть при тренуванні не достатньо її було і тому іноді використовує англійські слова, кальку і не може речення і суть передати правильно.

Відповіді чат-боту (крок 8):

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Окрім оцінювання за рубрикою, використовують автоматичні бенчмарки, тестування на наборі завдань із відомими відповідями, перевірку згенерованого коду тестами, оцінювання експертами та порівняння відповідей моделями-суддями. Також вимірюють швидкість генерації, затримку першого токена, вартість запитів, стабільність результатів і кількість фактичних помилок.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Моделі часто навчені оформлювати відповіді у зручному для читання вигляді, тому можуть додавати Markdown-обгортки або вступний текст навіть за прямої заборони. Для надійнішого отримання JSON можна використовувати структурований режим виведення, якщо API його підтримує, та перевіряти результат через json.loads().

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Навчальні дані багатьох моделей містять значно більше англійськомовних текстів, тому модель краще засвоює англійську лексику, граматику та стилістичні конструкції. У менших моделей також менше параметрів для засвоєння мовних закономірностей, через що вони частіше допускають русизми, кальки й неприродні формулювання українською.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Генерація може бути недетермінованою через параметри семплювання, наприклад temperature, тому модель обирає різні продовження тексту. Через це відповіді можуть відрізнятися за точністю, форматом і повнотою, навіть якщо запит залишається незмінним.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — це ситуація, коли модель генерує неправдиву, вигадану або непідтверджену інформацію та подає її як достовірну. Модель прогнозує ймовірне продовження тексту, а не перевіряє автоматично кожне твердження за джерелами, тому за недостатнього контексту може сформувати переконливу, але неправильну відповідь.

Висновок за завданням 5

Оцінювання за рубрикою виявилося корисним для порівняння моделей, оскільки дозволило оцінити фактологічну точність, дотримання формату, якість переказу, працездатність коду, логічне мислення та володіння українською мовою. Водночас результат залежить від чіткості критеріїв і правильності перевірки відповідей, тому для надійнішої оцінки потрібно проводити більше прогонів.

Найпоказовішими виявилися завдання на логічні обчислення та українську мову, адже в них добре помітні помилки в міркуваннях, суперечності й мовні недоліки. Для роботи з українськомовними текстами, як і для будь-якого завдання, я обрав би Gemini 3.5 Flash Lite, оскільки в цьому експерименті вона отримала найвищу оцінку — 12 із 12 балів — і продемонструвала найкращу якість відповідей. Проте для важливих завдань її результати також потрібно перевіряти і пам’ятати що це не приватна(локальна), а хмарна модель і дані прямують до Google.

Завдання 6. Керування випадковістю: temperature і top_p

Перевірка як значення temperature і top_p впливають на випадковість і унікальність відповідей. Були проведені 10 запусків з temperature = 0, щоб переконатися, що відповідь не змінюється.

Для 2 варіанту:

Творче завдання: Назва застосунку нагадувань про ліки

Фактичне питання: Рік відкриття Київського метрополітену

Таблиця 10: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	-
temperature	1.5	5	1*
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

* рік правильно, але не дата (но так як запитання саме про рік, то зарахую як +)

Таблиця 11: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на запитання (крок 7):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Я встановив би temperature = 0.0, оскільки для генерації JSON важливі передбачуваність, стабільність і дотримання заданої структури, а не різноманітність відповідей. Проте навіть за такого значення потрібно перевіряти результат за допомогою json.loads() та валідації потрібних полів.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора заголовків я вибрав би temperature = 1.0–1.5, оскільки в творчих завданнях корисно отримувати різноманітні формулювання та ідеї. Моя таблиця показує, що за temperature = 1.5 вдалося отримати п’ять унікальних відповідей, хоча збільшення температури також може погіршити їхню якість, особливо у старих моделях як llama3.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Параметр temperature змінює розподіл імовірностей токенів: низькі значення роблять вибір більш зосередженим на найімовірніших токенах, а високі — більш різноманітним. top_p натомість обмежує набір кандидатів для вибору токена найменшою групою, сумарна ймовірність якої досягає заданого порога.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні, temperature = 0 не гарантує абсолютної відтворюваності за будь-яких умов, оскільки на результати можуть впливати реалізація моделі, обчислення на обладнанні та параметри сервісу. У моєму експерименті всі 10 запусків дали одну унікальну відповідь, тому в межах цього тесту результат був відтворюваним, але це не буде відбуватися завжди.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з’явитися.

Відповідь:

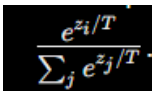
'Колісно правильно! Київське metRO відкрито в □□□ 1966, а на moment його відкриття було лише одним з перших Київскї metropolів.'

(там арабські символи, які не відображаються у Ворді)

При такому високому значенні температури, розподіл імовірностей токенів стає більш рівномірним і збільшує шанси менш імовірних продовжень, які можуть виявитися нелогічними або не відповідати запитанню. Тому й з’являються китайські, японські, арабські, англійські символи й слова й незрозумілі конструкції.

Відповіді чат-боту (крок 8):

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Температура ділить оцінки токенів (логіти) на значення \(T\) перед застосуванням функції softmax:  .За низької температури ймовірності концентруються біля найімовірніших токенів, а за високої — розподіл стає більш рівномірним; за temperature = 0 зазвичай використовують вибір найімовірнішого токена.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Temperature змінює ймовірності всіх кандидатів, а top_p відбирає групу найімовірніших токенів, сукупна ймовірність яких досягає заданого порога, і виключає решту. Отже, температура змінює розподіл імовірностей, а top_p обмежує набір токенів, з яких модель може вибирати.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Змінювати обидва параметри одночасно можна, але для початку краще змінювати один, залишаючи інший незмінним, щоб зрозуміти вплив кожного. Після цього можна перевірити їхні комбінації та порівняти різноманітність, правильність і стабільність відповідей.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Навіть за temperature = 0 на результат можуть впливати особливості обчислень на обладнанні, реалізація алгоритмів, налаштування сервісу або зміни версії моделі.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних, наприклад JSON, зазвичай використовують низьку температуру — 0.0–0.2, щоб зменшити варіативність відповідей. Для творчого тексту, генерації заголовків та ідей часто використовують 0.7–1.2 або вищі значення, якщо потрібна більша різноманітність; оптимальні параметри залежать від моделі та завдання.

Висновок за завданням 6

Для наступних лабораторних робіт, у яких модель повинна повертати структуровані дані, я оберу temperature = 0.0. Це допоможе зменшити випадковість результатів, але не замінить перевірку коректності JSON, наявності потрібних полів і правильності даних.

Для творчих завдань я обиратиму вищу температуру, оскільки вона допомагає отримувати різні варіанти відповідей. Експеримент показав, що вищі значення можуть збільшувати різноманітність, але не гарантують правильності. Налаштування за замовчуванням можуть бути непридатними, тому що вони не обов’язково відповідають меті конкретного завдання: для JSON важливі структура та точність, а для творчого тексту — різноманітність і оригінальність.

Підсумок

Таблиця 12: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	13 - 125	~ 23(точно не вимірювали)	38 - 862
TTFT	2.06 – 2.09	(не вимірювали)	0.64 – 0.69

Якість, балів /12	2	11	12
Стабільність формату (завдання 2)	JSON синтаксично коректний, але структура відповідей відрізнялася	JSON синтаксично коректний, структура відповідей стабільніша	JSON синтаксично коректний, структура відповідей стабільна
Потрібні ресурси	Відносно невеликі: локальна модель, оперативна пам’ять і ресурси процесора або відеокарти	Більші, ніж для 3B: більше пам’яті та обчислювальних ресурсів	Локальні обчислювальні ресурси для генерації не потрібні; потрібні пристрій і мережеве з’єднання
Вартість	Безкоштовно, лише обладнання і електрика	Безкоштовно, лише обладнання і електрика	Залежить від тарифу, є обмежений безкоштовний тариф
Куди йдуть ваші дані	На свій локальний комп’ютер	На свій локальний комп’ютер	До хмарних сервісів Google
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для дуже простих завдань, локального запуску, приватності та обмежених	Для локальних завдань, де потрібна краща якість, але дані бажано не	Для завдань, де важливі висока якість відповідей, грамотність українською та правильне

	ресурсів, де не потрібна висока якість	передавати в хмару	виконання складних інструкцій

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій моделей чи промтів не може ґрунтуватися на одиничних запусках через високу випадкову похибку результатів.